

SOMAFOAM

Model equations, numerical schemes
and boundary conditions

October 2026

This document describes what the solver `somaFoam` solves and how, as implemented in `src/plasmaCookBook` and `applications/somaFoam`.

Contents

1	Overview	1
2	Governing equations	2
2.1	Species: drift–diffusion	2
2.2	Species: momentum equation	2
2.3	Reactions	2
2.4	Poisson equation	2
2.5	Electron energy	3
2.6	Ion and gas temperature	3
2.7	Dielectric regions	3
3	Discretisation and available schemes	4
3.1	Flux schemes for drift–diffusion	4
3.2	Terms and scheme entries	4
3.3	Time stepping and inner iterations	5
3.4	Limits	5
3.5	Acceleration of slow neutral species	5
3.6	Adaptive mesh refinement	6
4	Boundary conditions	6
4.1	Potential	6
4.2	Electron density: <code>driftDiffusionElectronDensity</code>	7
4.3	Ion density	8
4.4	Electron temperature: <code>electronTemperature</code>	8
4.5	Auxiliary fields	8
5	Diagnostics	8

1 Overview

SOMAFOAM is a finite-volume fluid model of low-temperature plasmas built on foam-extend 4.0. For each time step it solves, in this order,

1. the Poisson equation for the potential Φ (with the dielectric regions, if any),

2. a continuity equation for every transported species, with a drift–diffusion flux or with a momentum equation,
3. the ion and gas temperatures, according to the selected models,
4. the electron energy equation for the electron temperature T_e .

Temperatures are in kelvin throughout, densities in m^{-3} , and all other quantities in SI units.

Symbol	Meaning
$n_k, \mathbf{\Gamma}_k$	number density and particle flux of species k
z_k	charge number (-1 for electrons)
μ_k, D_k	mobility (positive) and diffusion coefficient
T_k	temperature of species k (T_e for electrons, T_i for ions, T for the gas)
m_k, M	particle mass of species k and of the background gas
ν_k	momentum-transfer collision frequency
S_k	net production rate by the reactions
$\Phi, \mathbf{E} = -\nabla\Phi$	potential and electric field
ε	permittivity
σ	surface charge density on a dielectric surface
$\mathbf{n}$	unit normal on a boundary, pointing out of the plasma
δ	distance from the centre of the cell next to a boundary to the boundary face

2 Governing equations

2.1 Species: drift–diffusion

For species with `transportModel driftDiffusion` (models `driftDiffusion` and `mixed`),

$$\frac{\partial n_k}{\partial t} + \nabla \cdot \mathbf{\Gamma}_k = S_k, \quad \mathbf{\Gamma}_k = \mu_k \left(z_k \mathbf{E} - \frac{k_B}{e} \nabla T_k \right) n_k - D_k \nabla n_k. \quad (1)$$

The first term of the flux is the drift in the field together with the thermal-diffusion drift; the code calls the bracket times μ_k the drift velocity $\mathbf{F}_k$. Neutral species have $\mathbf{\Gamma}_k = -D_k \nabla n_k$.

The mobility and the diffusion coefficient of each species are set by `mobilityModel` and `diffusionModel` in `constant/plasmaProperties`:

Entry	Meaning
<code>constant</code>	constant value given in the species dictionary
<code>eTemp</code>	table of μN or DN against T_e (<code>constant/mu_<species></code> , <code>D_<species></code>), divided by the gas number density N
<code>EON</code>	the same, tabulated against the reduced field E/N
<code>einsteinRelation</code>	(diffusion only) $D_k = \mu_k k_B T_k / e$

2.2 Species: momentum equation

For species with `transportModel momentum` (models `momentum` and `mixed`) the velocity $\mathbf{u}_k$ is solved for:

$$\frac{\partial n_k}{\partial t} + \nabla \cdot (n_k \mathbf{u}_k) = S_k, \quad (2)$$

$$\frac{\partial (n_k \mathbf{u}_k)}{\partial t} + \nabla \cdot (n_k \mathbf{u}_k \mathbf{u}_k) + \nu_k n_k \mathbf{u}_k = \frac{1}{m_k} \left[-\nabla (n_k k_B T_k) + z_k e n_k \mathbf{E} \right]. \quad (3)$$

The collision frequency is $\nu_k = e / (\mu_k m_k)$ with `collisionFrequency muBased`, or is taken from the reaction rates with `rrBased`.

2.3 Reactions

The reactions are listed in `constant/reactions`. Rate coefficients are of Arrhenius form or tabulated against T_e (files `constant/reaction_*`); electron-impact reactions carry the energy lost by the electron per event. The net rate S_k of each species is linearised in its own density,

$$S_k \simeq S_k^* + \left(\frac{\partial S_k}{\partial n_k} \right)^* (n_k - n_k^*), \quad (4)$$

where $*$ denotes the previous inner iteration, and the derivative is treated implicitly.

2.4 Poisson equation

Selected by `poissonEquationSolver` in `constant/electroMagnetics`.

explicit

$$\nabla \cdot (\varepsilon \nabla \Phi) = -e \sum_k z_k n_k. \quad (5)$$

semiImplicit The space charge at the new time level is predicted from the species equations, and the part of the prediction that depends on the new field is moved to the left-hand side:

$$\nabla \cdot \left[\left(\varepsilon + \Delta t e \sum_k |z_k| \mu_k n_k \right) \nabla \Phi \right] = -e \sum_k z_k \left[n_k^o + \Delta t \left(S_k + \nabla \cdot (D_k \nabla n_k) + \nabla \cdot \left(\mu_k \frac{k_B}{e} n_k \nabla T_k \right) \right) \right], \quad (6)$$

with n_k^o the density at the old time level. This removes the dielectric-relaxation limit on the time step. A third entry, **linearized**, solves the explicit equation and reconstructs the field from the face fluxes of the potential.

2.5 Electron energy

Model `efullImplicit` (entry `energyModel { electron ...; }`):

$$\frac{\partial}{\partial t} \left(\frac{3}{2} k_B n_e T_e \right) + \nabla \cdot \left(\frac{5}{2} k_B T_e \mathbf{\Gamma}_e \right) - \nabla \cdot (\kappa_e \nabla T_e) = -e \mathbf{\Gamma}_e \cdot \mathbf{E} - Q_{\text{inel}} - Q_{\text{el}}. \quad (7)$$

- Conductivity: $\kappa_e = \frac{5}{2} k_B^2 n_e T_e / (m_e \nu_e)$, which with `muBased` is $\frac{5}{2} k_B^2 n_e T_e \mu_e / e$.
- Q_{inel} : energy lost in the electron-impact reactions, from the reaction list.
- $Q_{\text{el}} = 3k_B \frac{m_e}{M} \nu_e n_e (T_e - T)$: elastic losses to the gas (with `muBased`).
- The losses are linearised in T_e in the same way as the reaction sources.

Consistent fluxes. The electron flux in (7) is the flux of the electron continuity equation *through the cell faces*, exactly as that equation discretises it. With Γ_f the electron flux through face f (positive out of the cell) and Φ_f the potential interpolated to the face, the convective term uses $\frac{5}{2} k_B \Gamma_f$ and the heating in a cell of volume V is

$$-e (\mathbf{\Gamma}_e \cdot \mathbf{E})_P = e (\mathbf{\Gamma}_e \cdot \nabla \Phi)_P = \frac{e}{V} \sum_f \Gamma_f (\Phi_f - \Phi_P), \quad (8)$$

which is the discrete form of $\nabla \cdot (\mathbf{\Gamma} \Phi) - \Phi \nabla \cdot \mathbf{\Gamma}$. The energy the field delivers and the energy the electrons receive then match cell by cell. The fluxes of the species in the cells (used for the currents) are reconstructed from the face fluxes. For electrons with the momentum model the flux $n_e \mathbf{u}_e$ at the cell centres is used instead.

2.6 Ion and gas temperature

Entry	Model	Equation
ion	gasTemp	$T_i = T$
	gasCorrectedTemp	$T_i = T + \frac{M_i + M}{5M_i + 3M} \frac{M(\mu_i \mathbf{E})^2}{k_B}$ (Wannier)
gas	none	T constant
	gfullImplicit	steady conduction, $-\nabla \cdot (\kappa \nabla T) = \sum_{\text{ions}} e \mathbf{\Gamma}_k \cdot \mathbf{E} + \text{ion-gas exchange}$

2.7 Dielectric regions

With `solutionDomain plasmaDielectric` each dielectric region solves $\nabla \cdot (\varepsilon_d \nabla \Phi) = 0$. Charge accumulates on the interface,

$$\frac{\partial \sigma}{\partial t} = \mathbf{J} \cdot \mathbf{n}, \quad \mathbf{J} = e \sum_k z_k \mathbf{\Gamma}_k, \quad (9)$$

and the potential satisfies $\varepsilon_d \partial_n \Phi_d - \varepsilon \partial_n \Phi = \sigma$ there (see Section 4.1).

3 Discretisation and available schemes

3.1 Flux schemes for drift–diffusion

`fluxScheme` in `constant/plasmaProperties` selects how the drift–diffusion flux (1) and the convection–conduction flux of (7) are discretised. It can also be given inside the dictionary of a species, for that species alone.

fvSchemes (default). Drift and diffusion are separate terms, discretised with the convection and Laplacian schemes named in `system/fvSchemes` (Table 1).

scharfetterGummel. Drift and diffusion are one flux. Between the centres P and N of two neighbouring cells, a distance d apart, with u the drift velocity normal to the face and A the face area,

$$\Gamma_f = \frac{DA}{d} \left[B(-\text{Pe}) n_P - B(\text{Pe}) n_N \right], \quad \text{Pe} = \frac{u d}{D}, \quad B(x) = \frac{x}{e^x - 1}. \quad (10)$$

This is the exact solution of the steady one-dimensional problem between the two centres. It reduces to central differencing for $|\text{Pe}| \ll 1$ and to upwind differencing for $|\text{Pe}| \gg 1$, and gives non-negative densities without a limiter. The same form is used for the energy equation with $u \rightarrow \frac{5}{2} k_B \Gamma_f$ and $D \rightarrow \kappa_e$. Boundary faces other than processor and cyclic faces are treated as with `fvSchemes`. There is no non-orthogonal correction.

Written as separate terms, the scheme is central differencing for diffusion with the convective face value $n_f = w n_P + (1 - w) n_N$, $w = 1/(1 - e^{-\text{Pe}}) - 1/\text{Pe}$.

3.2 Terms and scheme entries

Table 1: Terms, their entries in `system/fvSchemes`, and the schemes that can be used.

Term	Entry	Schemes
Time derivatives	<code>ddtSchemes default</code>	<code>Euler</code> (first order, implicit; used in all examples)
Drift of charged species, $\nabla \cdot (\mathbf{F}n)$	<code>div(F,Ni)</code>	<code>Gauss + upwind</code> , <code>linear</code> , <code>limitedLinear 1</code> , <code>vanLeer</code> , <code>MUSCL</code> , <code>Minmod</code> , <code>SuperBee</code> , <code>vanAlbada</code> , <code>OSPRE</code> , <code>UMIST</code> , <code>QUICK</code> , <code>Gamma</code> , <code>limitedCubic</code> ; or <code>fluxScheme</code> <code>scharfetterGummel</code>
Diffusion of charged species	<code>laplacian(D,Ni)</code>	<code>Gauss linear + orthogonal</code> , <code>corrected</code> , <code>limited ψ</code> , <code>uncorrected</code> ; not used with <code>scharfetterGummel</code>
Diffusion of neutral species	<code>laplacian(D,Nin)</code>	as above
Convection of electron energy	<code>div((interpolate(eeFlux)&S),Te)</code>	as <code>div(F,Ni)</code> ; not used with <code>scharfetterGummel</code> for the electrons
Electron heat conduction	<code>laplacian(eC,Te)</code>	as <code>laplacian(D,Ni)</code>
Poisson equation	<code>laplacian(eps,Phi)</code>	<code>Gauss linear + orthogonal</code> or <code>corrected</code>
Thermal-diffusion term in (6)	<code>laplacian(D,T)</code>	as above
Momentum model: convection of density and velocity	<code>div(U,Ni)</code> , <code>div(flux,Ui_h)</code>	convection schemes as <code>div(F,Ni)</code>
Gas temperature	<code>laplacian(alpha,T)</code>	Laplacian schemes
Gradients ($\mathbf{E} = -\nabla\Phi$, ∇T)	<code>gradSchemes default</code>	<code>Gauss linear</code>

Notes.

- `limitedLinear 1` is the scheme of the examples. With consistent energy fluxes it and `vanLeer` give results within a few per cent of Scharfetter–Gummel on the meshes tested (1 Torr argon, 200 cells).
- On meshes that are not orthogonal, including the fine/coarse interfaces of adaptively refined 2D and 3D meshes, use `corrected` Laplacian and `snGrad` schemes.
- On processor boundaries the face values are interpolated between the two cells as on internal faces, so parallel and serial results agree.

3.3 Time stepping and inner iterations

One PIMPLE iteration per time step is the normal setting. Within a time step each species equation and the energy equation are solved repeatedly to resolve the linearised sources, as set in `constant/plasmaProperties`:

innerIterations entry	Default	Meaning
chargedSpecies	4	passes per charged species
neutralSpecies	2	passes per neutral species
electronTemperature	6	passes of the energy equation
tolerance	10 ⁻⁵	initial residual that ends the passes
reportUnconverged	yes	report loops that end above the tolerance

The time step is fixed when `deltaTMin` and `deltaTMax` in `system/controlDict` are equal; otherwise it follows $\Delta t = \text{MaxCo} / \max(\frac{1}{2}|\nabla \cdot \mathbf{F}_e|, \dots)$ between those bounds. Field relaxation factors `<field>Final` in `system/fvSolution` apply on the last PIMPLE iteration.

3.4 Limits

Floors and ceilings are applied after each solve (`limits` in `constant/plasmaProperties`; they are read again when the file changes during a run): `densityFloor` (10⁴ m⁻³), `TeMin` (300 K), `TeMax` (none), and per field `N_<species> { min; max; }, Te, Tion, T`.

3.5 Acceleration of slow neutral species

With `slowSpeciesAcceleration`, after every `fullCycles` periods of the applied voltage the listed neutral species are advanced alone over a long time Δt_a per step:

$$\frac{n_k^{\text{new}} - n_k}{\Delta t_a} - \nabla \cdot (D_k \nabla n_k^{\text{new}}) = \langle S_k \rangle - \left\langle \frac{\partial S_k}{\partial n_k} \right\rangle n_k + \left\langle \frac{\partial S_k}{\partial n_k} \right\rangle n_k^{\text{new}}, \quad (11)$$

where $\langle \cdot \rangle$ is the average over the last `averageCycles` periods. With `steadyState yes` the time derivative is dropped. The change per advance is limited by `maxChangeFactor` and `relaxation`.

3.6 Adaptive mesh refinement

Cells are refined where an indicator exceeds `lowerRefineLevel` and merged where it is below `unrefineLevel`. The indicator of a cell is the maximum over the listed entries:

type	Value
relativeGradient	$h \nabla f / (f + \text{floor})$
gradient	$ f_1 - f_2 / \text{scale}$ between neighbouring cells 1 and 2
magnitude	$ f / \text{scale}$

Here h is the size of the cell. With $L = |f| / |\nabla f|$ the local scale length of the field, `relativeGradient` is h/L : a cell is refined when it is larger than `lowerRefineLevel` times the scale length, and each level halves h . The `floor` keeps regions where f is negligible from being refined.

1D meshes use `dynamicRefine1DFvMesh`; 2D and 3D meshes use `dynamicPolyRefinementFvMesh` with the `plasmaIndicatorRefinement` selection. Merged cells take the volume average of the fields.

4 Boundary conditions

On a boundary face, n_b and T_b are the face values and n_P , T_P the values in the adjacent cell. Several conditions are of mixed type,

$$\phi_b = f \phi_{\text{ref}} + (1 - f) (\phi_P + \delta g_{\text{ref}}), \quad (12)$$

with weight f , reference value ϕ_{ref} and reference gradient g_{ref} . $E_n = \mathbf{E} \cdot \mathbf{n}$ is positive when the field points into the wall, that is, when electrons are repelled from it.

4.1 Potential

Type	Condition
<code>fixedValue</code>	$\Phi = \text{value}$ (grounded electrode)
<code>plasmaPotential</code>	driven electrode, by <code>model</code> : <code>directCurrent</code> : $\Phi = A$; <code>cosFrequencyModulated</code> : $\Phi = A \cos(2\pi ft) + V_{\text{bias}}$; <code>sinFrequencyModulated</code> : $\Phi = A \sin(2\pi ft) + V_{\text{bias}}$ (entries <code>amplitude</code> , <code>frequency</code> , <code>bias</code>)
<code>externalCircuitPotential</code>	electrode behind a series resistance R and capacitance C : $\Phi = A \sin(2\pi ft) + V_{\text{bias}} + IR + Q_C/C$, with I the total current through the patch and $Q_C = \int I dt$ (entries <code>R</code> , <code>C</code>). With <code>directCurrent</code> the source is $A(1 - e^{-t/\tau})$, with $\tau = \text{rampTime}$ (no ramp if absent)
<code>rfPowerElectrode</code>	RF electrode at a given mean power behind a blocking capacitor: $\Phi = A g(t) + V_{\text{dc}}$. <i>Waveform</i> g (<code>waveform</code>): <code>sine</code> , $\sin(2\pi ft)$; <code>harmonics</code> , $\sum a \sin(2\pi nft + \varphi)$ over the entries (n a φ) of <code>harmonics</code> , φ in degrees; <code>table</code> , linear interpolation in <code>waveformTable</code> over the phase $ft \bmod 1$. <i>Power</i> . Every period the mean current $\langle I \rangle$ and mean power $\langle \Phi I \rangle$ through the patch are evaluated (I is the total current into the plasma). The target P is <code>power</code> , or <code>powerTable</code> ($(t \ P) \dots$) interpolated in time, repeated with <code>powerRepeat</code> . <code>tracking feedback</code> : every <code>updateCycles</code> periods $A \leftarrow A(P/\langle \Phi I \rangle)^{\gamma/2}$, with $\gamma = \text{relaxation}$, limited to the fraction <code>maxChange</code> , and $A \leftarrow A\sqrt{P_{\text{new}}/P_{\text{old}}}$ when the target changes. <code>tracking learning</code> (repeated table): one amplitude per RF period b of the table and a common gain, $A = g a_b$. Each time period b has been run, $a_b \leftarrow a_b(P/\langle \Phi I \rangle)^{\lambda/n_b}$ and $g \leftarrow g(P/\langle \Phi I \rangle)^{\gamma_g/2}$, with $\lambda = \text{learningRelaxation}$, $\gamma_g = \text{gainRelaxation}$ and $n_b = \Delta \ln \langle \Phi I \rangle / \Delta \ln A$ from the last two passes of that period (2 at first). $A \leq \text{amplitudeMax}$; no RF voltage where $P = 0$. <i>Blocking capacitor</i> (<code>blockingCapacitor</code>): <code>ideal</code> , $V_{\text{dc}} \leftarrow V_{\text{dc}} - \langle I \rangle / (fC)$ every period ($C = \text{capacitance}$ acts as a gain); <code>physical</code> , $V_{\text{dc}} = V_{\text{bias}} - Q/C$ with $Q = \int I dt$; <code>none</code> , fixed bias
<code>dielectricSideWall</code>	charging wall without a meshed dielectric: $\partial_n \Phi = \sigma / \varepsilon_0$ (no field inside the wall), with σ from (9)
<code>coupledPotential</code>	plasma–dielectric interface, <code>somaFoam</code> : the two regions are solved in one matrix; the interface potential is $\Phi_w = \frac{\varepsilon_1 \Phi_1 / \delta_1 + \varepsilon_2 \Phi_2 / \delta_2 + \sigma}{\varepsilon_1 / \delta_1 + \varepsilon_2 / \delta_2}$
<code>iterativeCoupledPotential</code>	plasma–dielectric interface, <code>plasmaMultiRegionFoam</code> : the plasma side fixes Φ_w , the dielectric side imposes $\partial_n \Phi_d = (\sigma - \varepsilon \partial_n \Phi) / \varepsilon_d$ (normals out of each region), and Φ_w is relaxed (Aitken) until both agree

4.2 Electron density: `driftDiffusionElectronDensity`

The condition imposes the net electron flux to the wall: the thermal flux, always, plus the drift where it is directed to the wall, minus the emitted electrons,

$$\mathbf{\Gamma}_e \cdot \mathbf{n} = \frac{1}{4} \bar{v}_e n_b + \max(\mathbf{F}_e \cdot \mathbf{n}, 0) n_b - \Gamma_{se} - \Gamma_{FE}, \quad \bar{v}_e = \sqrt{\frac{8k_B T_e}{\pi m_e}}, \quad (13)$$

where $\mathbf{F}_e \cdot \mathbf{n} = -\mu_e(E_n + \frac{k_B}{e} \partial_n T_e)$ is the drift velocity towards the wall. It is implemented as a mixed condition with

$$f = \frac{C_1}{C_1 + D_e/\delta}, \quad C_1 = \frac{1}{4} \bar{v}_e + \max(-\mathbf{F}_e \cdot \mathbf{n}, 0), \quad n_{ref} = 0, \quad g_{ref} = \frac{\Gamma_{se} + \Gamma_{FE}}{D_e}. \quad (14)$$

C_1 is never negative, so $0 \leq f < 1$ for any wall cell. With `Edepend false` the flux is the thermal flux alone, or the drift to the wall where that is larger: $C_1 = \max(\frac{1}{4} \bar{v}_e - \mathbf{F}_e \cdot \mathbf{n}, 0)$.

- Secondary emission: $\Gamma_{se} = \gamma \max(\mathbf{\Gamma}_i \cdot \mathbf{n}, 0)$, with $\gamma = \text{seec}$ and $\mathbf{\Gamma}_i$ the total ion flux.
- Field emission (`field_emission true`): the Fowler–Nordheim current density for the field βE_n (`field_enhancement_factor` β , `work_function` in eV), where $E_n > 0$.

4.3 Ion density

Type	Condition
<code>zeroGradient</code>	ions leave with their drift flux, $\mathbf{\Gamma}_i \cdot \mathbf{n} = \mu_i E_n n_b$ (used in the examples)
<code>driftDiffusionPositiveIonDensity</code>	as above where $E_n \geq 0$; zero net flux where the field points away from the wall: mixed with $f = C_1/(C_1 + D_i/\delta)$, $C_1 = -\mu_i E_n$

Neutral species use `zeroGradient` (no wall loss) or `fixedValue`.

4.4 Electron temperature: `electronTemperature`

The electron energy flux to the wall is the enthalpy carried by the electrons (Lymberopoulos and Economou, J. Phys. D **28** (1995) 727),

$$\mathbf{q}_e \cdot \mathbf{n} = \frac{5}{2} k_B T_e \Gamma_{out} - \frac{5}{2} k_B (\Gamma_{se} T_{se} + \Gamma_{FE} T_{FN}), \quad (15)$$

with Γ_{out} the flux of plasma electrons to the wall. The energy equation carries $\frac{5}{2} k_B T_e \mathbf{\Gamma}_e \cdot \mathbf{n} - \kappa_e \partial_n T_e$ through the wall face, and $\mathbf{\Gamma}_e \cdot \mathbf{n} = \Gamma_{out} - \Gamma_{se} - \Gamma_{FE}$ (13), so the condition on the temperature is

$$\kappa_e \partial_n T_e = -h (T_b - T_{em}), \quad h = \frac{5}{2} k_B (\Gamma_{se} + \Gamma_{FE}), \quad T_{em} = \frac{\Gamma_{se} T_{se} + \Gamma_{FE} T_{FN}}{\Gamma_{se} + \Gamma_{FE}}. \quad (16)$$

It is a mixed condition with

$$f = \frac{h}{h + \kappa_e/\delta}, \quad T_{ref} = T_{em}, \quad g_{ref} = 0. \quad (17)$$

h is never negative, so $0 \leq f < 1$ for any size of the cell next to the wall; without emission the condition is a zero gradient. Entries: `seec`, `Tse`, `TFN`, `field_emission`, `field_enhancement_factor`, `work_function` (`Edepend` is accepted and not used).

The value $\frac{5}{2} k_B T_e$ per electron is the one consistent with the convective term of (7). With the one-way kinetic value $2k_B T_e$ the difference would have to be returned by conduction with a gradient proportional to the wall temperature, which is unstable on coarse wall cells.

4.5 Auxiliary fields

The files `0/eeFlux` and `0/F_electron` hold boundary values of the thermal energy flux and thermal velocity (`eEnergyThermalFlux`, `electronThermalFlux`); they are needed for the fields to be created, and are used by the energy equation only for electrons with the momentum model.

5 Diagnostics

The `electrodeVoltageCurrent` function object writes, per patch,

$$I_{\text{cond}} = \int \mathbf{J} \cdot d\mathbf{S}, \quad I_{\text{disp}} = \int \varepsilon \frac{\partial \mathbf{E}}{\partial t} \cdot d\mathbf{S}, \quad I_{\text{tot}} = I_{\text{cond}} + I_{\text{disp}}, \quad (18)$$

with the sign reversed so that a current is positive from the electrode into the plasma. The `fieldAverage` function object with `resetOnOutput true` and a write interval of whole periods gives period-averaged fields.